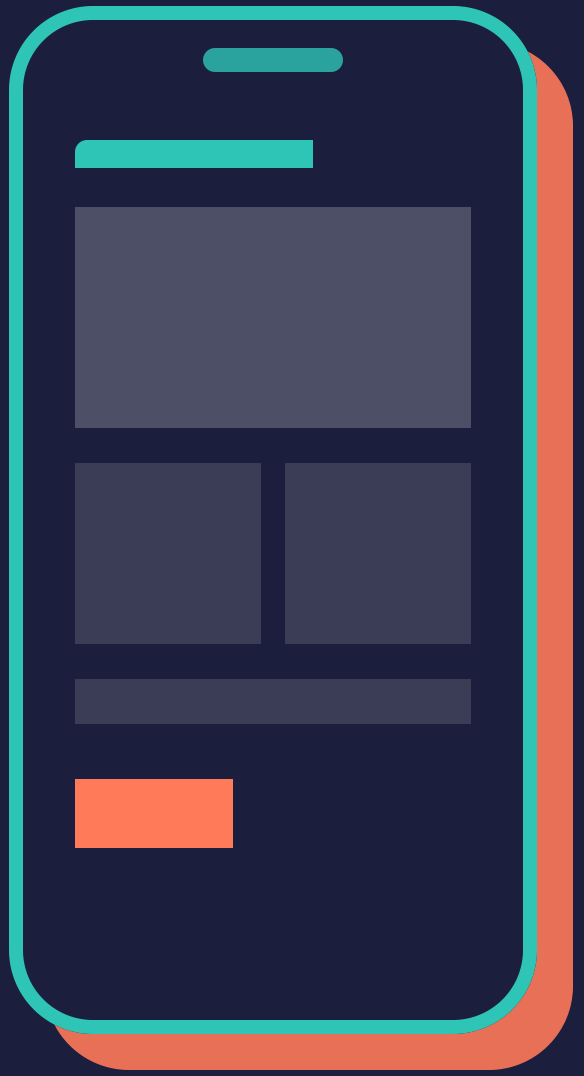
Fondations de React Native

Cours 1 – Architecture, web, composants et responsive

Chris CHEVALIER

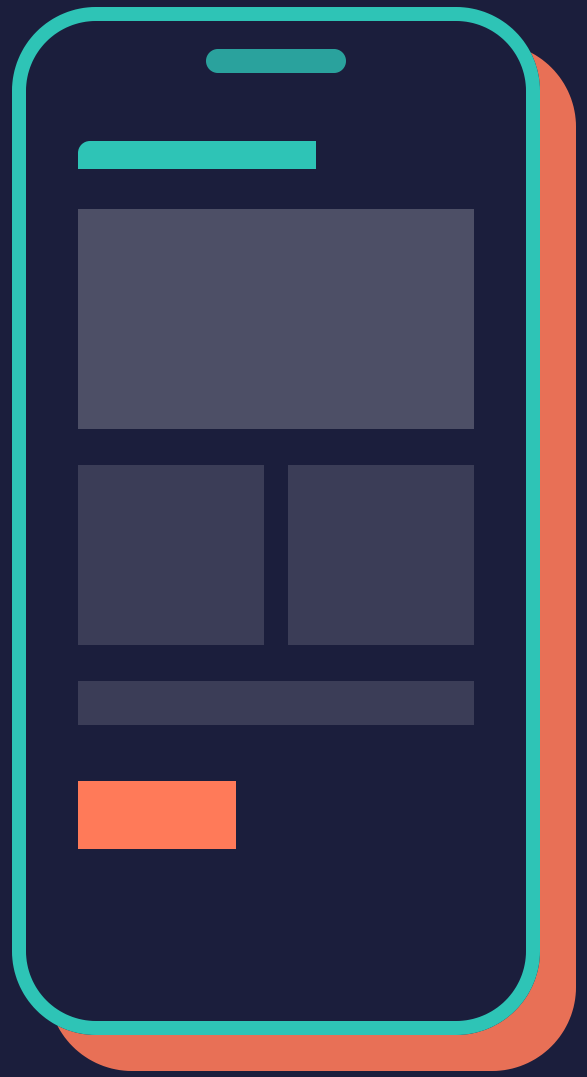


Sommaire

01 De React à React Native

02 Expo et TypeScript

03 Composants, styles et responsive



01

De React à React Native

Ce que vous savez déjà, ce qui change



Le même modèle, une autre cible

	React (web)	React Native
Ce que vous écrivez	Des composants, des props, de l'état, des hooks	
Les briques	`<div>`, `<span>`, `<img>`, `<input>`	`<View>`, `<Text>`, `<Image>`, `<TextInput>`
Les styles	CSS, cascade, sélecteurs	Objets JavaScript, aucune cascade
Le rendu	Le DOM du navigateur	Des vues natives iOS et Android (et le DOM sur le web)
La navigation	Des URL	Une pile d'écrans (et des URL sur le web)

> Votre logique React se réutilise telle quelle. Ce qui change, ce sont les **briques d'affichage** et les **contraintes du mobile**.

Comment JavaScript devient une vue native

1. Votre code

Vos composants TypeScript sont exécutés par **Hermes**, le moteur JavaScript optimisé pour le mobile

2. Le rendu

React calcule l'arbre des composants. Le moteur de rendu **Fabric** le traduit en vues natives

3. L'écran

iOS affiche une `UIView`, Android une `android.view.View`, le navigateur une `<div>`

Avant : le bridge

JavaScript et le natif échangeaient des messages **asynchrones** sérialisés en JSON. Le pont devenait un goulot d'étranglement.

Aujourd'hui : la nouvelle architecture

JSI (JavaScript Interface) permet des appels **directs** entre JavaScript et le natif. C'est l'architecture par défaut des projets actuels.

React Native for Web

- React Native for Web réimplémente les composants de React Native avec **React DOM**
- `<View>` devient une `<div>`, `<TextInput>` un `<input>`
- `StyleSheet` génère des **classes CSS** optimisées
- Un seul code pour trois plateformes

Ce que vous écrivez

```
<View style={styles.card}>
  <Text>Studio lumineux</Text>
</View>
```

Ce que le navigateur reçoit

```
<div class="css-view-g5y9jx r-…">
  <div dir="auto" class="css-text-…">
    Studio lumineux
  </div>
</div>
```

Quiz

Une application React Native peut-elle utiliser `<div>` ?

> Non. `<div>` n'existe que dans le DOM : l'application planterait sur iOS et Android. Utilisez `<View>`, que React Native for Web transforme en `<div>` dans le navigateur.

Une application React Native est-elle une page web dans une application ?

> Non. Sur mobile, ce sont de vraies vues natives. Une page web affichée dans une application s'appelle une **WebView** (c'est l'approche de Cordova ou d'Ionic).

02

Expo et TypeScript

Créer et lancer un projet



Expo : créer et lancer le projet

Terminal

```
npx create-expo-app@latest mon-app
cd mon-app
npm run reset-project
npx expo start
```

- **Expo** est le framework recommandé par l'équipe de React Native
- `w` ouvre la version web. Le QR code s'ouvre avec Expo Go sur votre téléphone
- À la question de `reset-project`, répondez `n` : l'exemple est supprimé au lieu d'être déplacé
- Chaque sauvegarde met l'écran à jour (Fast Refresh)

Un réglage pour le web. Dans `app.json`, remplacez `"output": "server"` par `"output": "single"`. Le rendu sur serveur ne connaît pas la taille de la fenêtre : un écran responsive y provoque une erreur d'**hydratation**.

Méfiez-vous des tutoriels anciens et des réponses d'IA : `expo eject` ou `expo-cli` installé globalement ont **disparu**.

Le téléphone et l'ordinateur doivent être sur le **même réseau**. Si le Wi-Fi de l'école bloque la connexion, lancez `npx expo start --tunnel`.

L'arborescence du projet

mon-app/

```
src/
├── app/              écrans et navigation (Expo Router)
│   ├── _layout.tsx   navigateur racine
│   └── index.tsx     écran de l'URL « / »
├── components/       composants réutilisables
├── hooks/            hooks personnalisés
├── models/           types des données
└── data/             données locales
assets/               images, polices
app.json              configuration Expo
tsconfig.json         configuration TypeScript
```

- **Chaque fichier de `src/app/` est un écran**. Son chemin donne son URL
- Tout le reste du code vit **en dehors** de `src/app/`
- L'alias `@/` pointe vers `src/` : `import { STAYS } from "@/data/stays"`

TypeScript : ce qui nous servira

src/models/stay.ts

```ts
export type StayType = "apartment" | "house" | "cabin";

export type Stay = {
  id: string;
  title: string;
  city: string;
  type: StayType;
  pricePerNight: number;
  available: boolean;
  imageUrl?: string;
};
```

- `type` décrit la **forme** d'une donnée
- `"apartment" | "house"` : une **union**, seules ces valeurs sont acceptées
- `imageUrl?` : propriété **facultative** (`string | undefined`)
- TypeScript vérifie dans l'éditeur et avec `npx tsc --noEmit` , **avant** l'exécution

03

Composants, styles et responsive

Les briques de toute interface



Les briques de base

Composant	Rôle	Équivalent web
`View`	Conteneur et mise en page	`<div>`
`Text`	Tout texte affiché	`<p>`, `<span>`
`Image`	Image locale ou distante	`<img>`
`TextInput`	Saisie de texte	`<input>`
`Pressable`	Zone qui réagit au toucher	`<button>`
`ScrollView`	Contenu qui défile (court)	`overflow: auto`
`FlatList`	Liste longue, affichée au fur et à mesure	Aucun

Tous s'importent depuis `react-native` : `import { View, Text } from "react-native";`

`View` et `Text`

src/components/greeting.tsx

```tsx
import { StyleSheet, Text } from "react-native";

type GreetingProps = { firstName: string; unread?: number };

export function Greeting({ firstName, unread = 0 }: GreetingProps) {
  return (
    <Text style={styles.greeting}>
      Bonjour <Text style={styles.bold}>{firstName}</Text>
      {unread > 0 && ` · ${unread} message(s)`}
    </Text>
  );
}

const styles = StyleSheet.create({
  greeting: { fontSize: 18 },
  bold: { fontWeight: "700" },
});
```

- **Tout texte doit être dans un `<Text>`**. Un texte posé dans une `<View>` fait planter l'application
- Un `<Text>` peut contenir d'autres `<Text>`. Ce sont les seuls à hériter du style de leur parent
- Dans un écran : `<Greeting firstName="Manon" unread={3} />`. Sans `firstName`, TypeScript refuse l'appel
- **Export nommé** pour un composant : `import { Greeting } from "@/components/greeting"`. Seuls les écrans de `src/app/` utilisent `export default`, exigé par Expo Router

Image

Deux sources d'image

```tsx
// Image locale, intégrée à l'application
<Image
  source={require("@/assets/images/icon.png")}
  style={{ width: 64, height: 64 }}
/>

// Image distante, téléchargée à l'affichage
<Image
  source={{ uri: stay.imageUrl }}
  style={{ width: "100%", aspectRatio: 4 / 3 }}
/>
```

- Une image locale passe par `require()` : elle est intégrée à l'application au moment du build
- Une image distante n'a **pas de taille par défaut** : sans largeur ni hauteur, elle reste invisible
- `aspectRatio` garde les proportions quand la largeur s'adapte à l'écran

`TextInput` : un champ contrôlé

src/components/name-form.tsx

```tsx
import { useState } from "react";
import { StyleSheet, Text, TextInput, View } from "react-native";

export function NameForm() {
  const [name, setName] = useState("");
  return (
    <View>
      <TextInput
        value={name}
        onChangeText={setName}
        placeholder="Votre prénom"
        style={styles.input}
      />
      <Text>{name ? `Bonjour ${name} !` : "…"}</Text>
    </View>
  );
}

const styles = StyleSheet.create({
  input: { borderWidth: 1, borderRadius: 8, padding: 10, marginBottom: 8 },
});
```

- Un champ **contrôlé**, comme en React : sa valeur vient de l'état
- `onChangeText` reçoit directement le texte, sans objet d'événement
- Équivalent web : `<input value onChange>` et `event.target.value`

Un retour visuel au toucher

stay-card.tsx (extrait)

```
<Pressable
  onPress={() => onToggleFavorite(stay.id)}
  style={({ pressed }) => [
    styles.button,
    pressed && styles.buttonPressed,
  ]}
>
  <Text style={styles.buttonLabel}>
    {isFavorite ? "★ Favori" : "☆ Ajouter aux favoris"}
  </Text>
</Pressable>
```

- Sur mobile, il n'y a pas de survol. L'utilisateur doit **voir** que son doigt a été pris en compte
- `style` accepte une fonction qui reçoit `{ pressed }`
- Un tableau de styles applique les suivants par-dessus les premiers. `false` est ignoré

Afficher une liste : `FlatList`

src/app/index.tsx (extrait)

```tsx
<FlatList
  data={visibleStays}
  keyExtractor={(stay) => stay.id}
  renderItem={({ item }) => (
    <StayCard
      stay={item}
      isFavorite={favorites.includes(item.id)}
      onToggleFavorite={toggleFavorite}
    />
  )}
  ListEmptyComponent={<Text>Aucun résultat</Text>}
/>
```

- `FlatList` n'affiche que les éléments **visibles** (virtualisation)
- `keyExtractor` joue le rôle de la prop `key` de `map()`
- `ScrollView` affiche tout d'un coup : réservez-la aux contenus courts

StyleSheet et Flexbox

Exemple de styles

```
const styles = StyleSheet.create({
  card: {
    flex: 1,
    backgroundColor: "#fff",
    borderRadius: 12,
    margin: 8,
  },
  header: {
    flexDirection: "row",
    justifyContent: "space-between",
    alignItems: "center",
  },
});
```

- Des propriétés CSS en **camelCase**, des nombres **sans unité**
- **Aucune cascade** : seul un `<Text>` transmet son style à un `<Text>` enfant
- Flexbox partout, mais `flexDirection` vaut `column` par défaut
- `flex: 1` : « prends toute la place disponible »
- `gap` espace les enfants sans marges

Quiz

Je place trois `<Text>` dans une `<View>` sans style. Comment s'affichent-ils ?

> En colonne, l'un sous l'autre : `flexDirection` vaut `column` par défaut.

Je donne `color: "red"` à une `<View>`. Le texte qu'elle contient devient-il rouge ?

> Non. Il n'y a pas de cascade : seul un `<Text>` parent transmet son style à un `<Text>` enfant.

Pas de media queries : un hook

src/hooks/use-columns.ts

```ts
import { useWindowDimensions } from "react-native";

// Nombre de colonnes selon la largeur de la fenêtre
export function useColumns(): number {
  const { width } = useWindowDimensions();
  if (width >= 1024) return 3;
  if (width >= 600) return 2;
  return 1;
}
```

- `StyleSheet` ne connaît pas les media queries du CSS
- `useWindowDimensions()` renvoie la taille de la fenêtre et **déclenche un nouveau rendu** quand elle change (rotation, redimensionnement)
- L'ancienne API `Dimensions.get()` ne se met pas à jour toute seule : préférez le hook

Une grille sur le web, une liste sur mobile

src/app/index.tsx (extrait)

```tsx
export default function CatalogScreen() {
  const columns = useColumns();
  // …
  return (
    <View style={styles.container}>
      <FlatList
        key={columns}
        numColumns={columns}
        data={visibleStays}
        keyExtractor={(stay) => stay.id}
        renderItem={renderStay}
      />
    </View>
  );
}
```

- `numColumns` transforme la liste en grille
- `FlatList` refuse de changer de nombre de colonnes à chaud. Changer sa `key` force un nouveau montage
- Sur le conteneur, `maxWidth: 1100` et `alignSelf: "center"` évitent une ligne de 2 500 px sur un grand écran

Démo : le catalogue de séjours

- Des données typées (`Stay`)
- Un composant `StayCard` aux props typées
- Une recherche avec `TextInput`
- Des favoris avec `Pressable` et `useState`
- Une grille de 1 à 3 colonnes selon la largeur

Je lance l'application sur le web puis sur mon téléphone avec Expo Go. Redimensionnez la fenêtre : la grille s'adapte en direct.

À retenir

- React Native affiche de **vraies vues natives**. React Native for Web les traduit en DOM
- **Expo** crée, lance et teste le projet. Chaque fichier de `src/app/` est un écran
- TypeScript décrit vos données et vos props, et vérifie avant l'exécution (`npx tsc --noEmit`)
- Tout texte est dans un `<Text>`. Les styles sont des objets, sans cascade, en colonne par défaut
- Le responsive passe par `useWindowDimensions()` et des points de rupture

> Place à la pratique : le **TP1** occupe les deux créneaux suivants.

Des questions ?

Chris CHEVALIER
chevalier@chris-freelance.com

